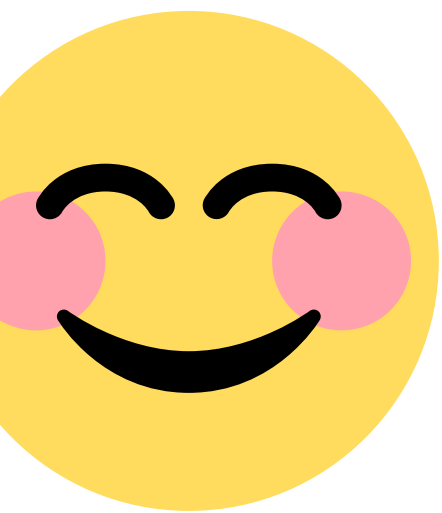




HOMEWORK

State Design Pattern

POWranger: Styve Alves

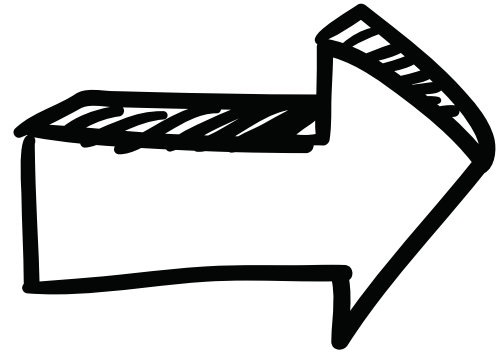


MARCH 6, 2023



DO IT !!





Design Patterns

A design pattern is a general repeatable solution to a commonly occurring problem in software design. It is a proven approach that has been developed and refined by experienced software engineers over time, to solve specific design problems in a reliable and efficient way.

By using design patterns, software developers can improve the quality, flexibility, and maintainability of their code, as well as reduce the time and effort required to develop complex software systems.

SCHEDULE

Introduction

Overview

Example_

Summary_

- **State Design Pattern**



Exercise Traffic light:



```
public class TrafficLight {  
  
    String currentstate = "green";  
    public void showLight(){  
  
        switch(currentstate){  
            case "green":  
                System.out.println("green light");  
                currentstate = "yellow";  
                break;  
            case "yellow":  
                System.out.println("yellow light");  
                currentstate = "red";  
                break;  
            case "red":  
                System.out.println("red lighth");  
                currentstate = "green";  
                break;  
        }  
    }  
}
```



Using State Design Pattern ex.



```
public class TrafficLight {  
  
    private State currentState;  
  
    public TrafficLight() {  
        currentState = new RedState(); // initial state is red  
    }  
  
    public void next() {  
        currentState = currentState.nextLight(this);  
    }  
  
    public void showLight(){  
        currentState.showLight();  
        next();  
    }  
  
}
```

```
public interface State {  
  
    State nextLight(TrafficLight trafficLight);  
  
    void showLight();  
}
```




```
public class GreenState implements State {

    @Override
    public State nextLight(TrafficLight trafficLight) {
        return new YellowState();
    }

    @Override
    public void showLight() {
        System.out.println("green");
    }

}
```

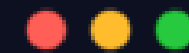


```
public class YellowState implements State {

    @Override
    public State nextLight(TrafficLight trafficLight) {
        return new RedState();
    }

    @Override
    public void showLight() {
        System.out.println("yellow");
    }

}
```



```
public class RedState implements State {

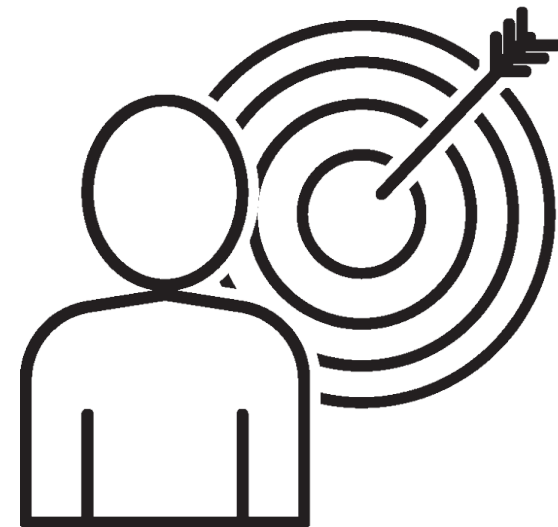
    @Override
    public State nextLight(TrafficLight trafficLight) {
        return new GreenState();
    }

    @Override
    public void showLight() {
        System.out.println("red!");
    }

}
```



```
public class Main {  
    public static void main(String[] args) {  
  
        TrafficLight trafficLight = new TrafficLight();  
        trafficLight.showLight();  
        trafficLight.showLight();  
        trafficLight.showLight();  
        trafficLight.showLight();  
  
    }  
}
```



finally
FINISHED

red!
green
yellow
red!

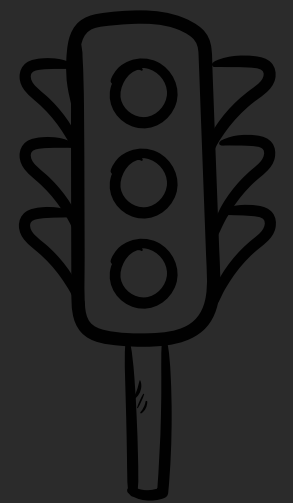
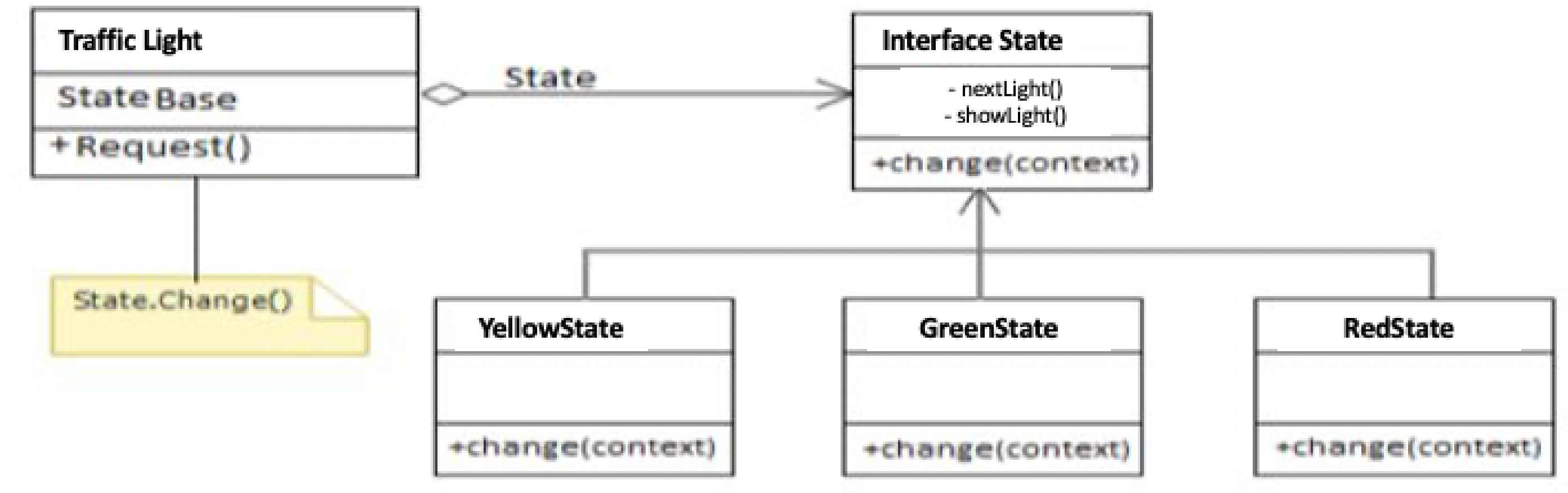
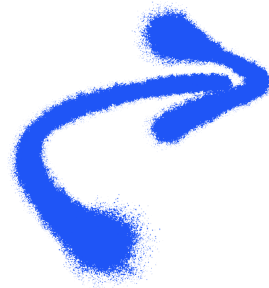




Diagram of the State Pattern





The State pattern consists of several key elements, including:

1. Context: the object that contains the state and whose behavior changes based on the state.
2. State: an interface that defines the methods and properties for the different states of the Context.
3. Concrete State: the implementations of the State interface, which define the behavior of the Context when it is in a specific state.



ATTENTION

YOUR

FOR

YOU

THANK

